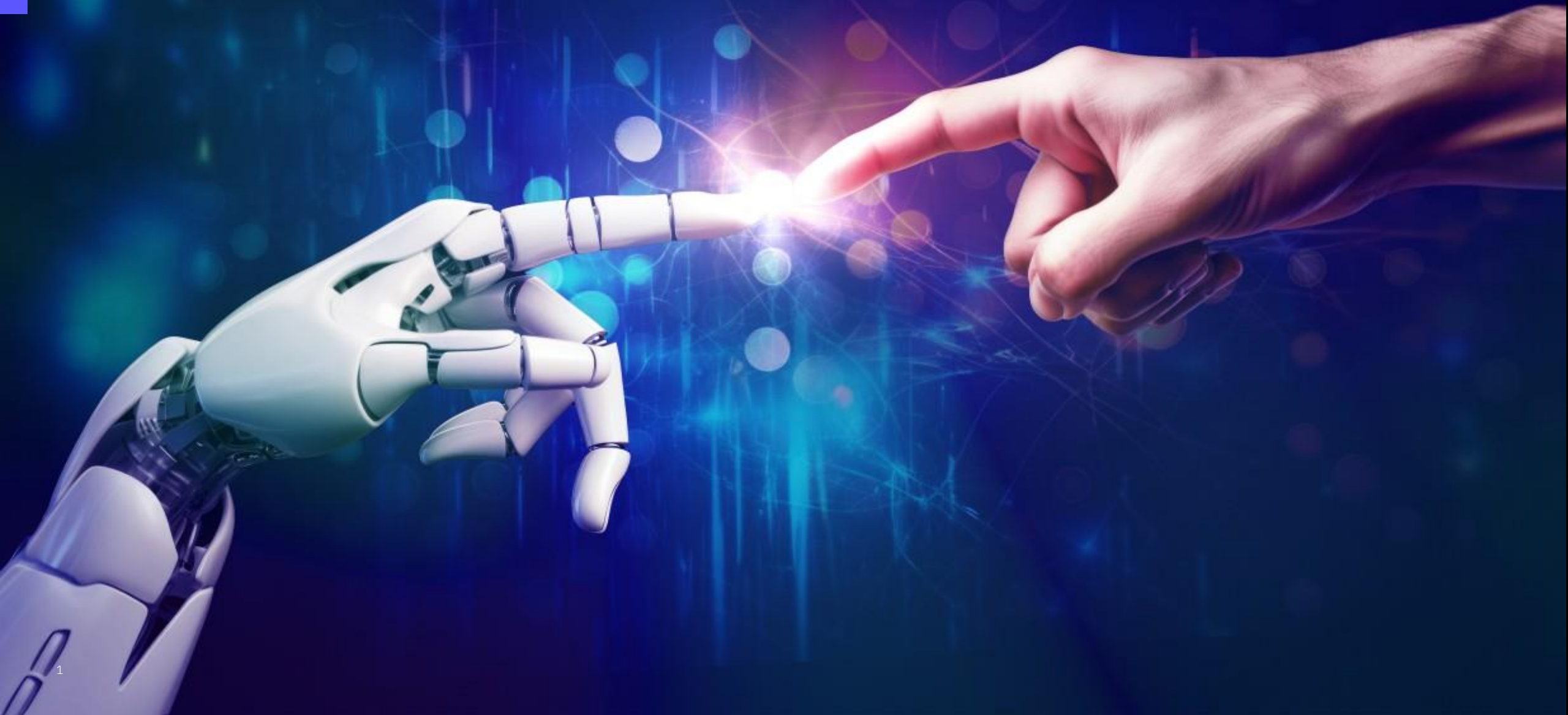


Generative AI Explained

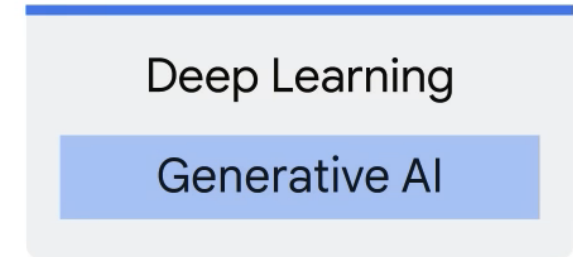
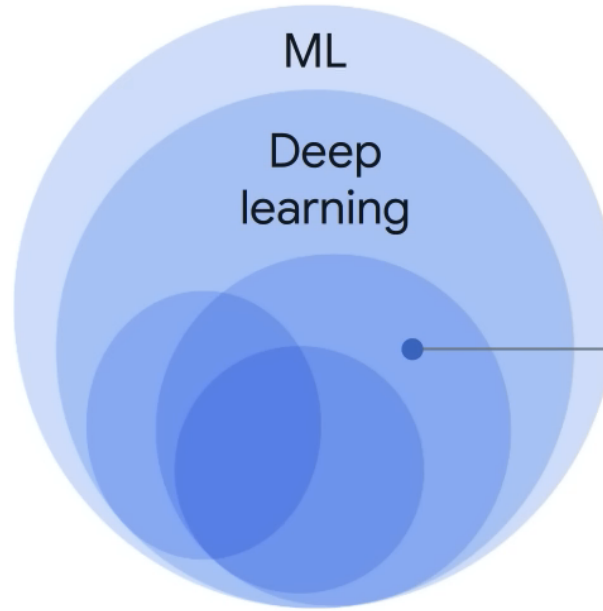


- ▶ 1. Overview of Gen AI
- 2. Attention is All you Need
- 3. It's About Cosine Similarity
- 4. What's Next for Gen AI
- 5. Is AI going to Replace Me?
- 6. Resources



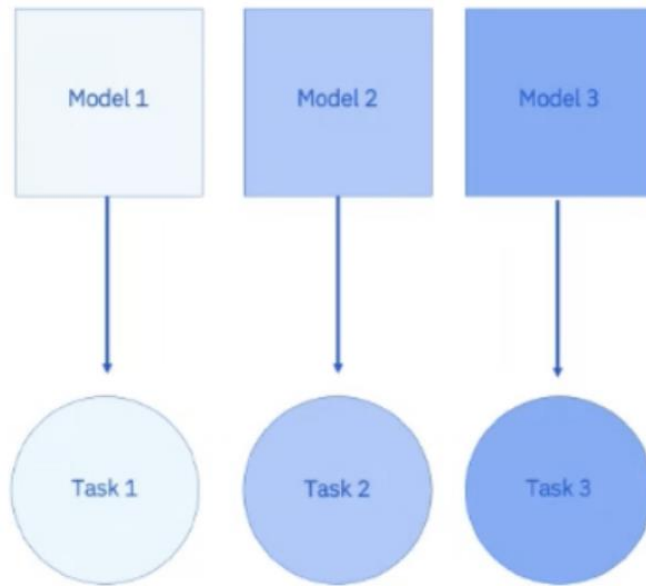
Generative AI

Generative AI
is a **subset of**
Deep Learning



Traditional AI Models

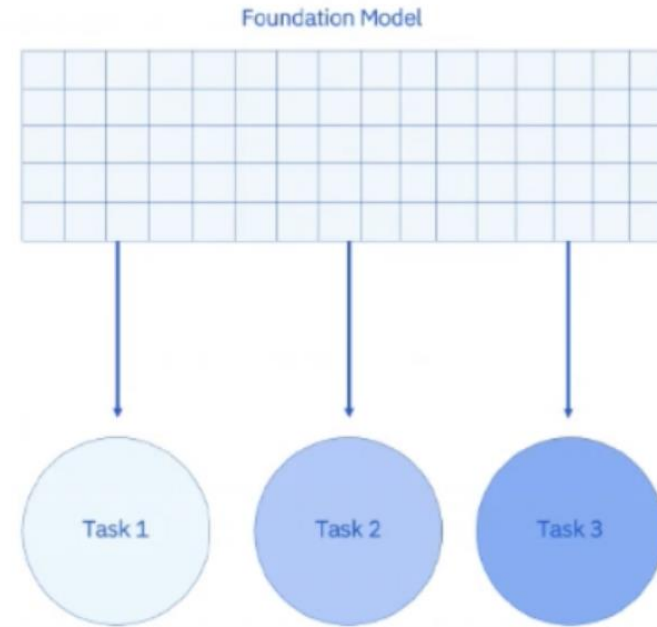
Each model is trained for a specific task



1,000s to 1,000,000s labeled data points per task

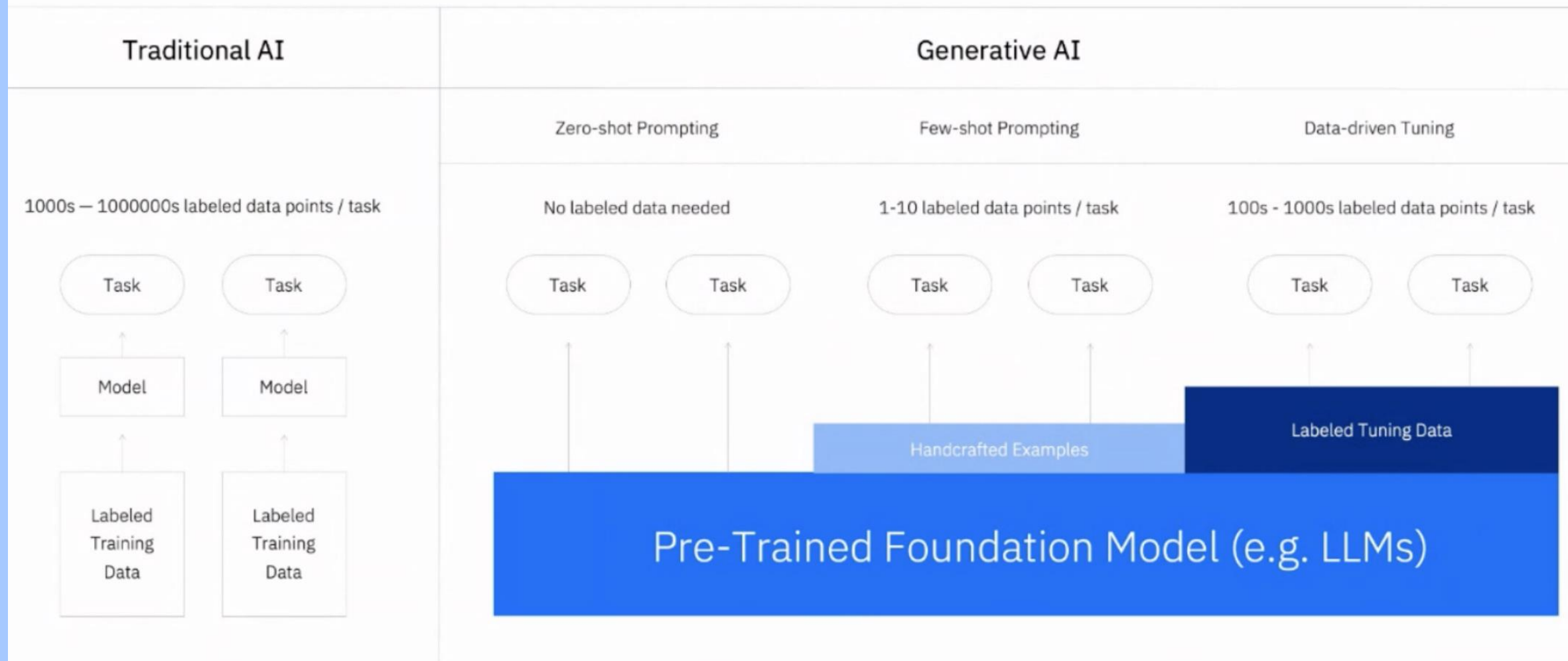
Foundation Models

One model that can address many tasks



0 to 1,000s labeled data points per task

A new paradigm for data-efficient development



Technique	Advantages	When to Use
Zero-Shot Prompting	- Simplest to implement - No training data needed	- Quickly test model capabilities - Simple tasks and inferences
One-Shot Prompting	- Can teach complex behaviors - Minimal training data	- Teaching nuanced or contextual responses - When only one good example is available
Few-Shot Prompting	- Teach more robust behaviors - Still very sample efficient	- Targeted enhancement for specific skills - When limited data for a task
Fine-Tuning	- Maximize model performance - Learn complex and nuanced behaviors	- Specializing model for dedicated tasks - When abundant training data exists
Parameter-Efficient Fine-Tuning	- Improved generalization - Faster and lower resource fine-tuning	- Specializing with limited data - Personalizing models for clients

THE EVOLUTION OF AI

.. an inflection point in AI

1980s

Big data



1990s–2000s

Labeled data
+
More compute



2010s

Big data
+
Powerful compute
+
Advanced algorithms



2017+

Self-supervision at scale
+
Massive unlabeled data
+
Compute at scale



TODAY

Foundation models
+
Multimodal
+
Agents & tools
+
Real-world impact



EXPERT SYSTEMS

Hand-crafted
symbolic
representations

Words: symbolic
Algorithms: rule-based / ML



MACHINE LEARNING

Task-specific
hand-crafted feature
representations

Words: symbolic / sparse
Algorithms: machine learning



DEEP LEARNING

Task-specific
learned feature
representations

Words: dense vectors
Algorithms: neural networks



SELF-SUPERVISED LEARNING

Learned from massive
unlabeled data

Words: dense vectors
Algorithms: deep neural
networks



FOUNDATION MODELS

Generalizable &
adaptable learned
representations

Words: dense vectors
Algorithms: transformer-
based models



FROM RULES TO REASONING. FROM TASK-SPECIFIC TO GENERAL PURPOSE. FROM DATA TO **IMPACT**.

1. Overview of Gen AI
- ▶ **2. Attention is All you Need**
3. It's About Cosine Similarity
4. What's Next for Gen AI
5. Is AI going to Replace Me?
6. Resources



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

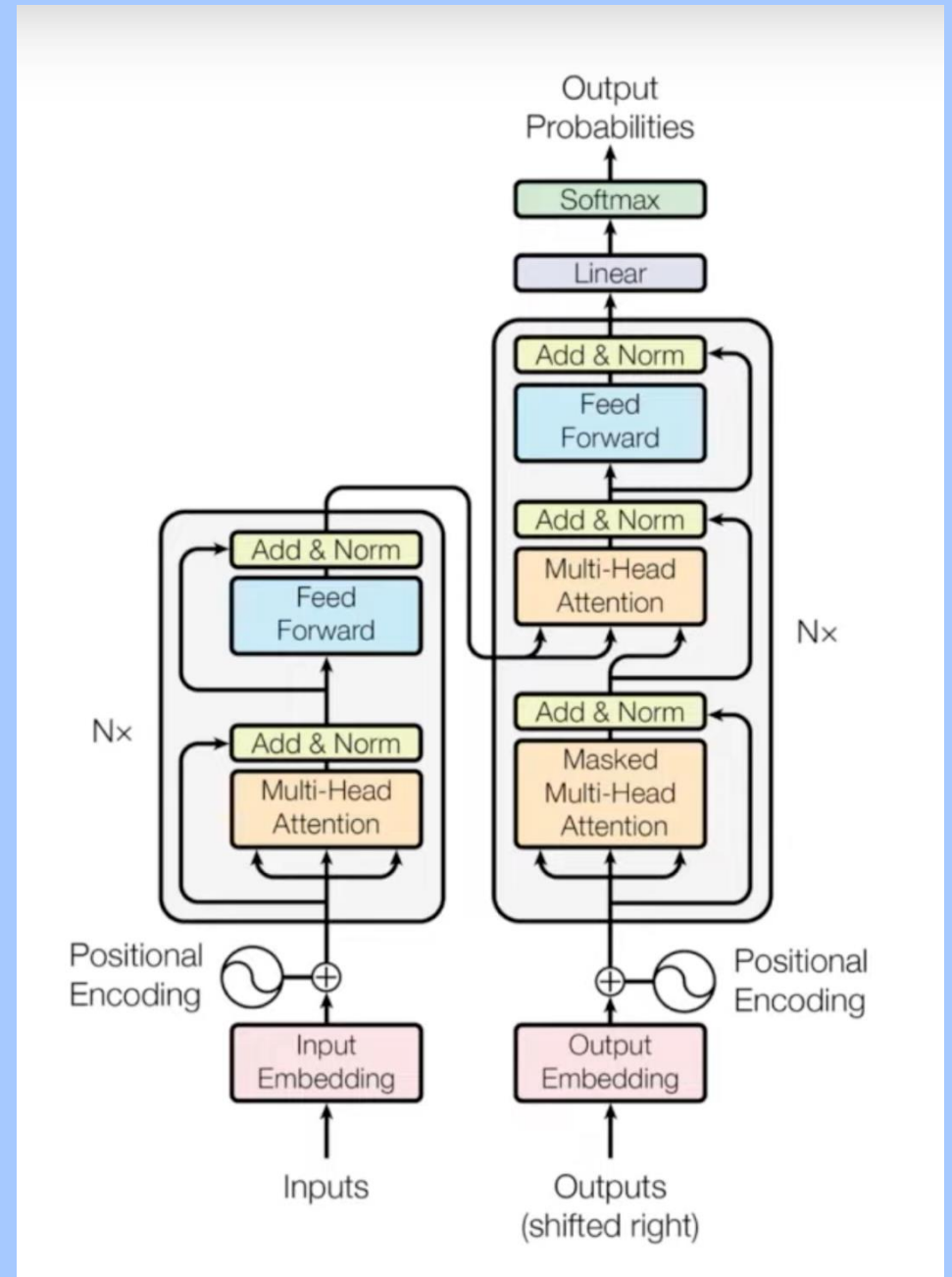
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Transformer architecture with: Encoder and Decoder uses: Attention Mechanism



Tokenization is the first thing that happens

Byte Pair Encoding (BPE): BPE is a subword tokenization algorithm that builds a vocabulary of variable-length subword units based on the frequency of their occurrences in the training data. It's widely used in NLP tasks.

WordPiece: Similar to BPE, WordPiece is a subword tokenization method. It starts with individual characters as tokens and iteratively merges them based on the likelihood of their co-occurrence in the training data. BERT, for example, uses the WordPiece tokenizer.

SentencePiece: SentencePiece is a more recent subword tokenization method that extends the BPE and WordPiece algorithms. It treats the input text as a sequence of sentences and segments it into a user-specified number of pieces. It has gained popularity in models like T5.

There are pluses and minuses

Word-based Tokenization

- Very large vocabularies
- A large quantity of out-of-vocabulary tokens
- Loss of meaning across very similar words

Character-based tokenization

- Very long sequences
- Less meaningful individual tokens

It's a balancing act



The goal is to efficiently represent the language's vocabulary with a manageable number of tokens.



Tokenizers are often tailored to the specific architecture of the model and may be variations of BPE or SentencePiece.



These methods are widely used in pre-training large language models because they efficiently handle the vast and diverse vocabulary present in natural language. They allow the models to represent and understand the complexity of language by breaking it down into meaningful subword units, collectively known as meaning spaces.

First, the sentence is tokenized and then comes embeddings!

Raw text: First, the sentence is tokenized and then comes embeddings!

Tokens ['First' , ' ' , 'the' , 'sen' , 'tence' , 'is' , ' ' , 'token' , 'ized' , 'and' , 'then' , 'come' , 's' , 'em' , 'bed' , 'ding' , 's' , '!']

Embeddings [203, 45, 1096, 77, 12, 92, 8, 423, 5096, 113, 97, 402, 420, 76, 19, 45]

Embeddings become **vector representations** and similar words gravitate towards each other in the vector **meaning space**.

Context matters, all the other words in the sentence help find meaningful words.

The smaller the **Cosine similarity**, greater the prediction.

You're in my meaning space

The model uses these numerical vectors as input to make predictions. During the generation phase, the model can also convert numerical vectors back into text, allowing it to generate coherent and contextually appropriate sentences.

This process enables the model to understand and generate text based on the learned relationships between tokens in the embedding space.

They allow the models to represent and understand the complexity of language by breaking it down into meaningful subword units, collectively known as **meaning spaces**.

It's how LLMs operate, capturing both syntactic and semantic information about language.

Fewer tokens means fewer computations, but it can also mean, less meaningful spaces!

It's a balancing act.

1. Overview of Gen AI
2. Attention is All you Need
- ▶ **3. It's About Cosine Similarity**
4. What's Next for Gen AI
5. Is AI going to Replace Me?
6. Resources



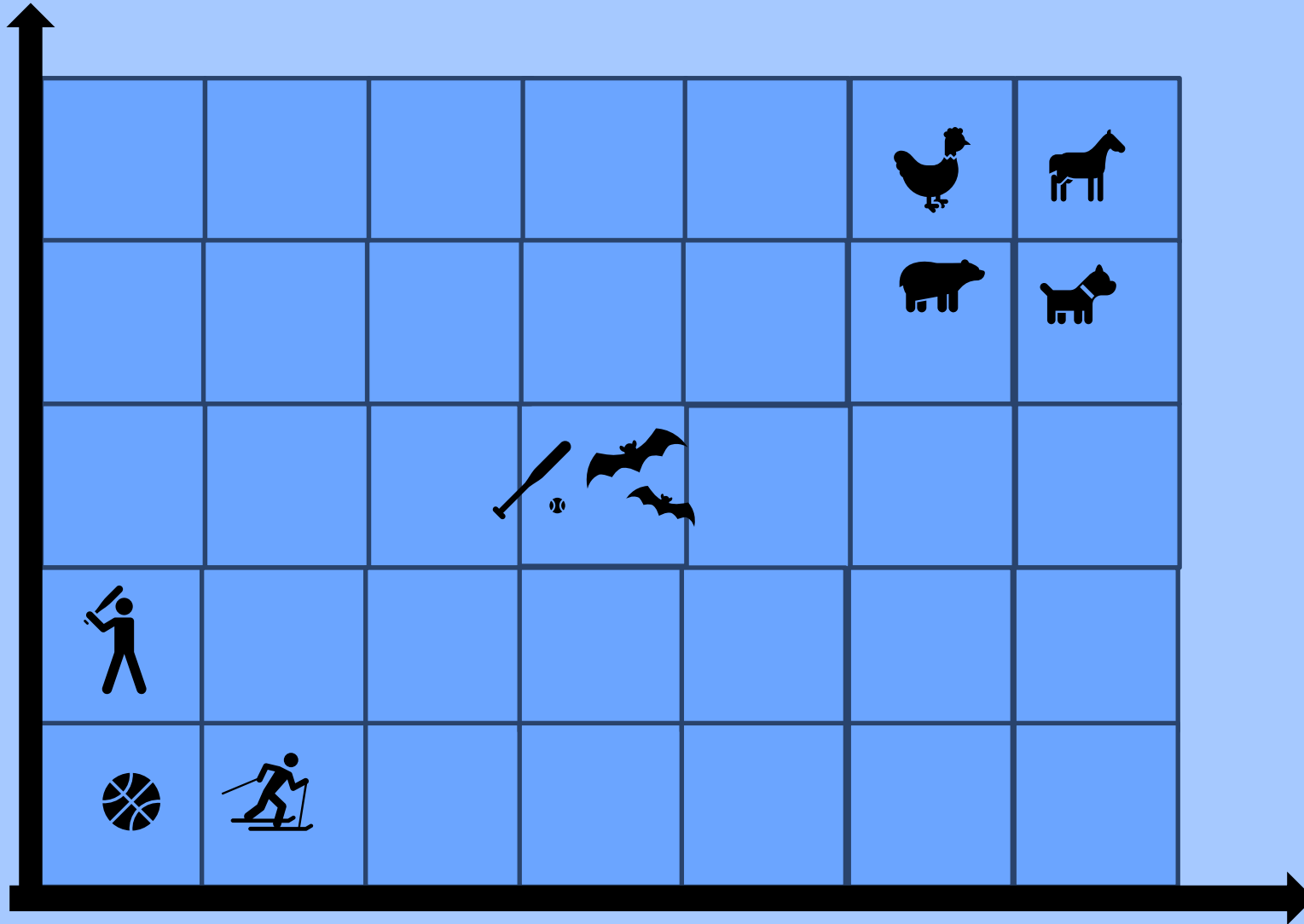
Similarity between words is
obtained using:

Dot-product

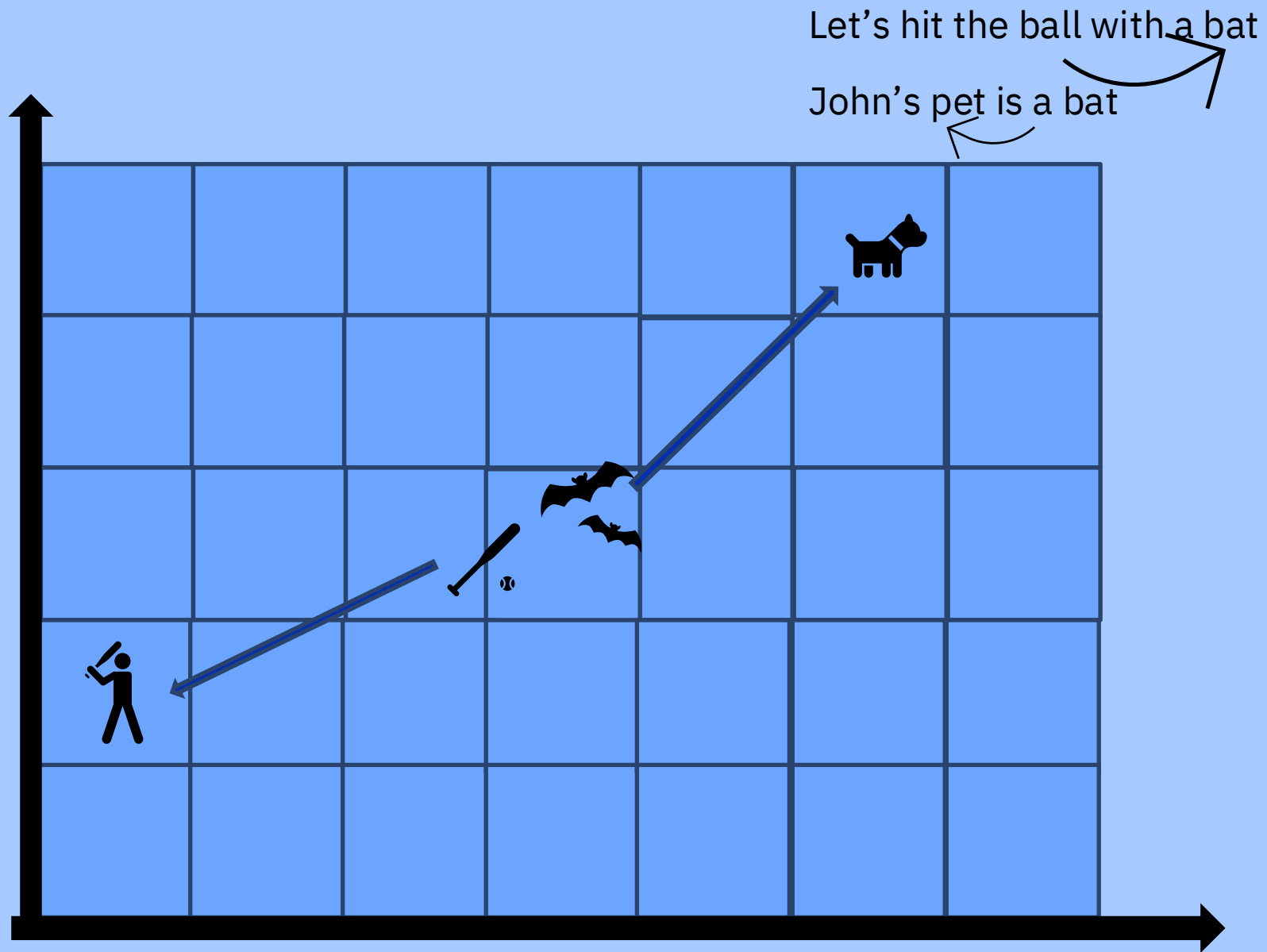
Cosine similarity

Scaled dot product

So how does it know which way to ‘pull’ **bat**?



Context is important

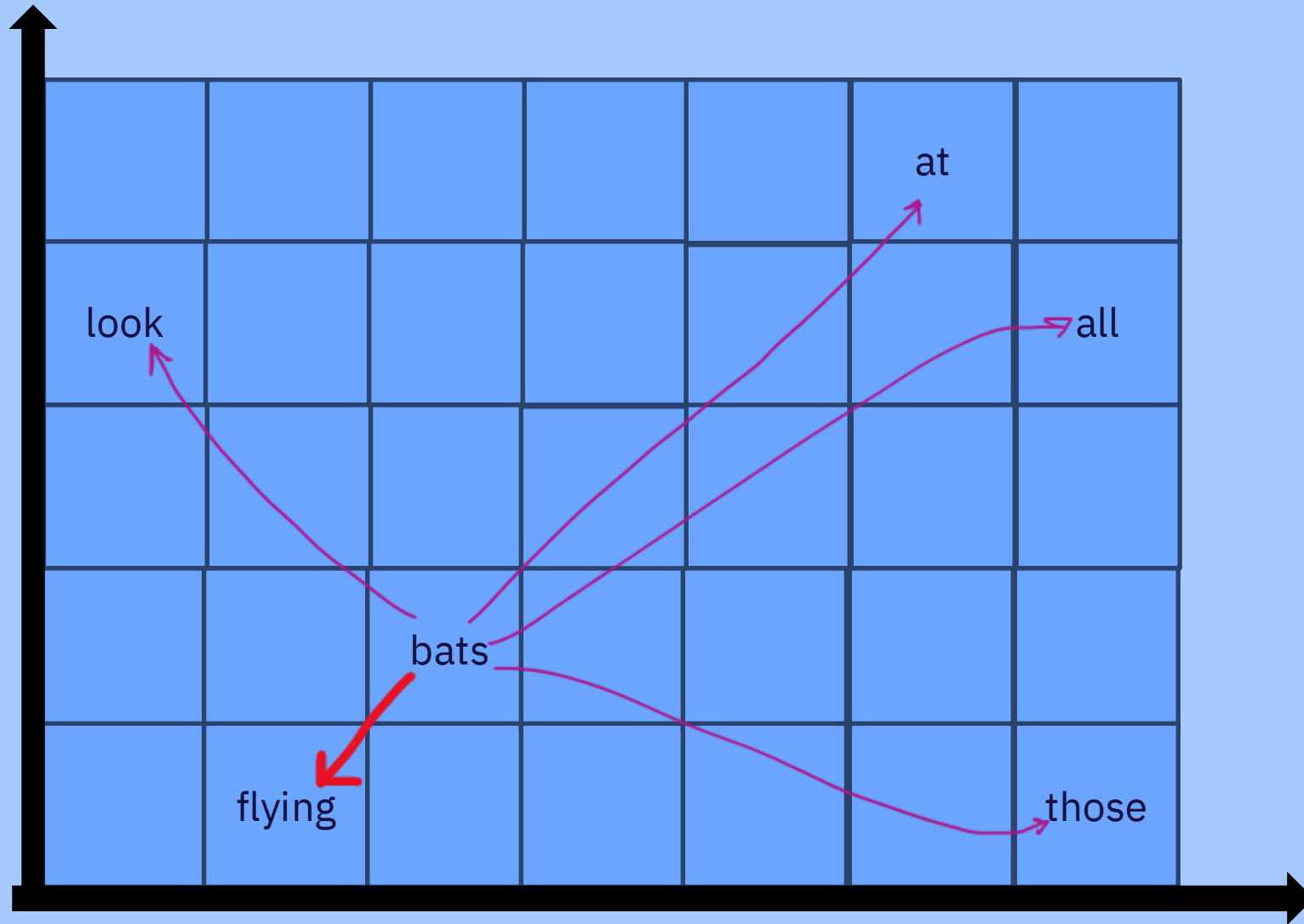


All the words are also pulling on bats

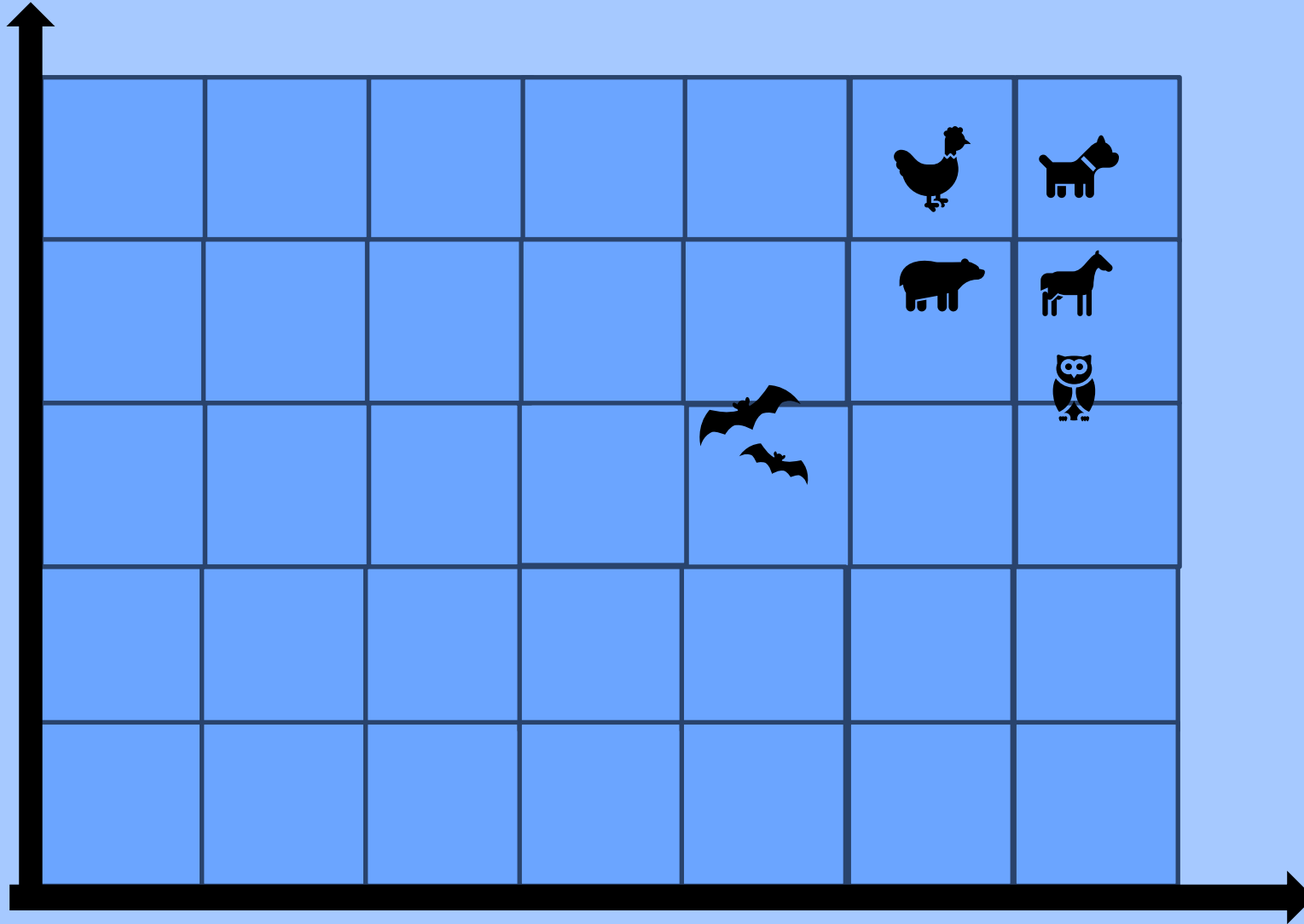
Look at all those bats flying

A diagram illustrating the concept of 'pulling on bats' using the sentence "Look at all those bats flying". Red arrows originate from the words "Look", "at", "all", "those", and "flying", and all point towards the word "bats". This visualizes the idea that every word in the sentence exerts a pull or influence on the word "bats".

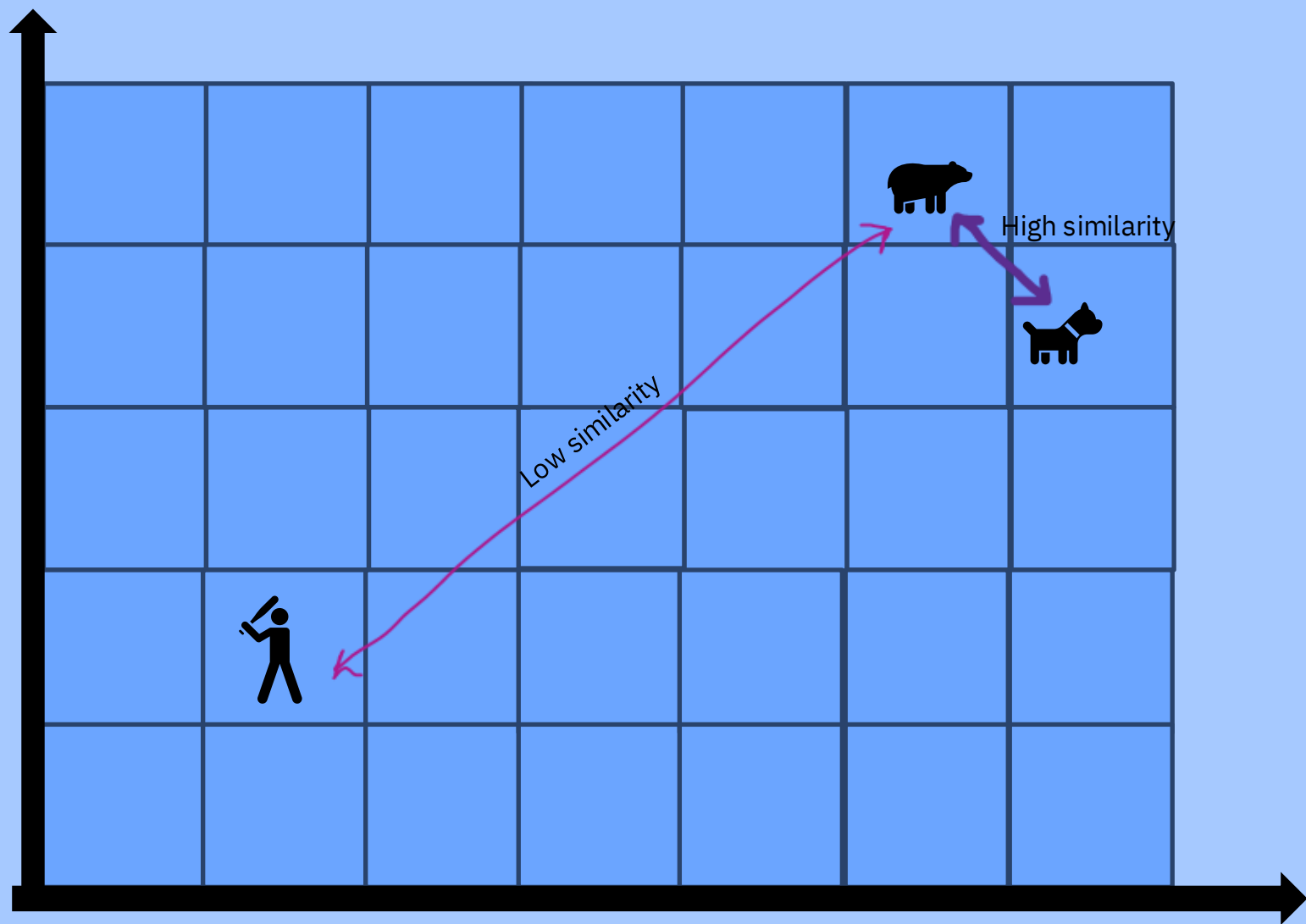
Some of the words are dis-similar, not as strong of a pull as the word flying



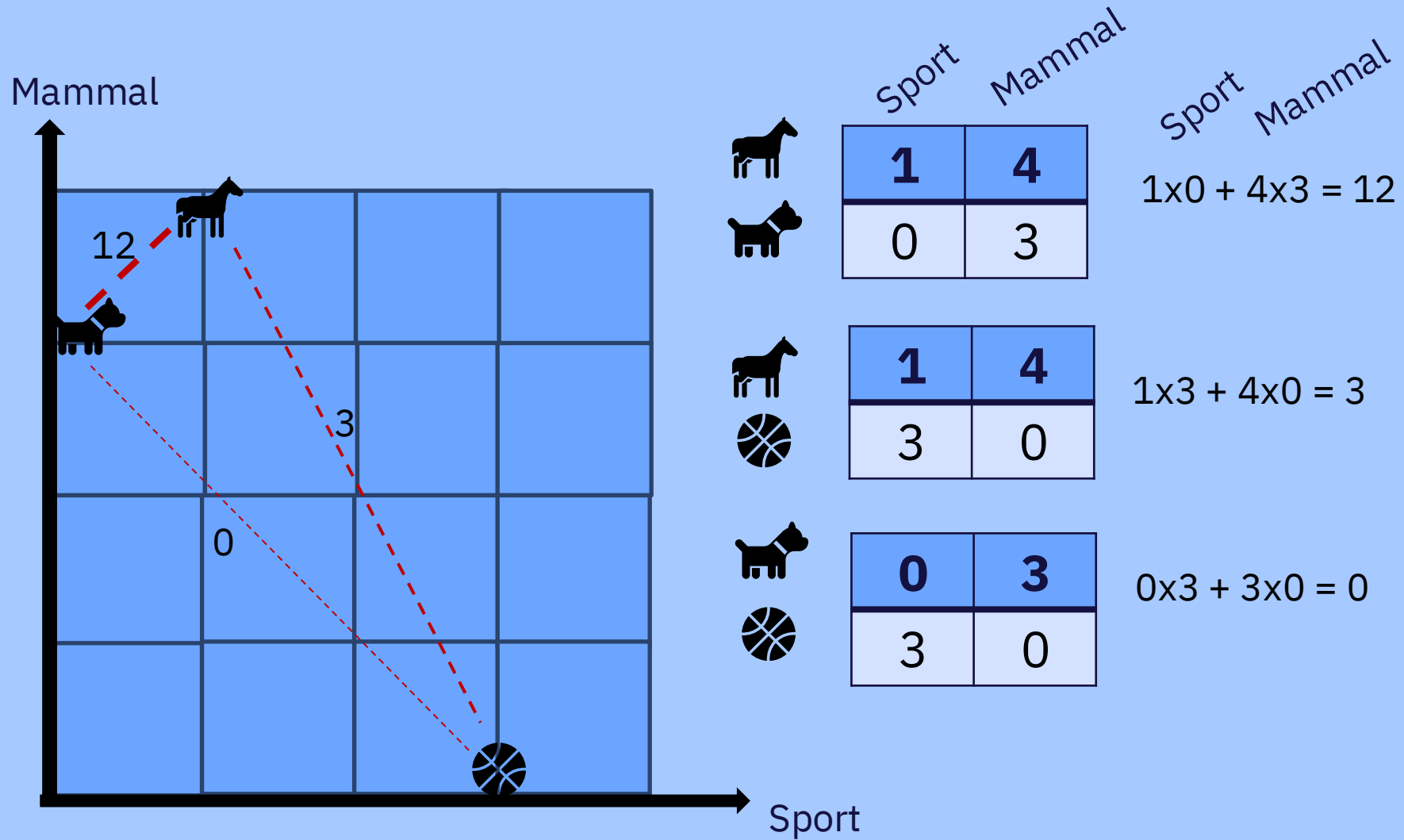
Context pulls



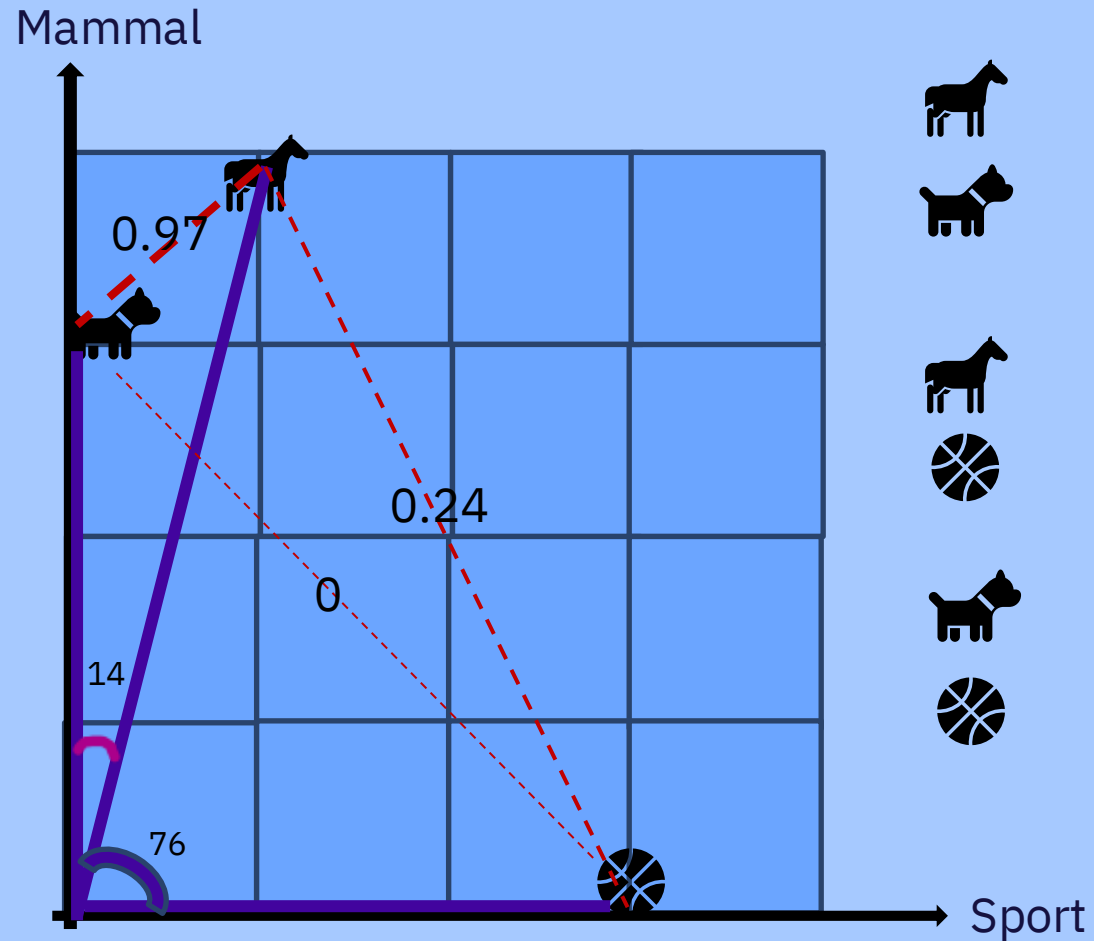
Let's talk about measure of similarity



Dot Product Measure



Cosine similarity measure

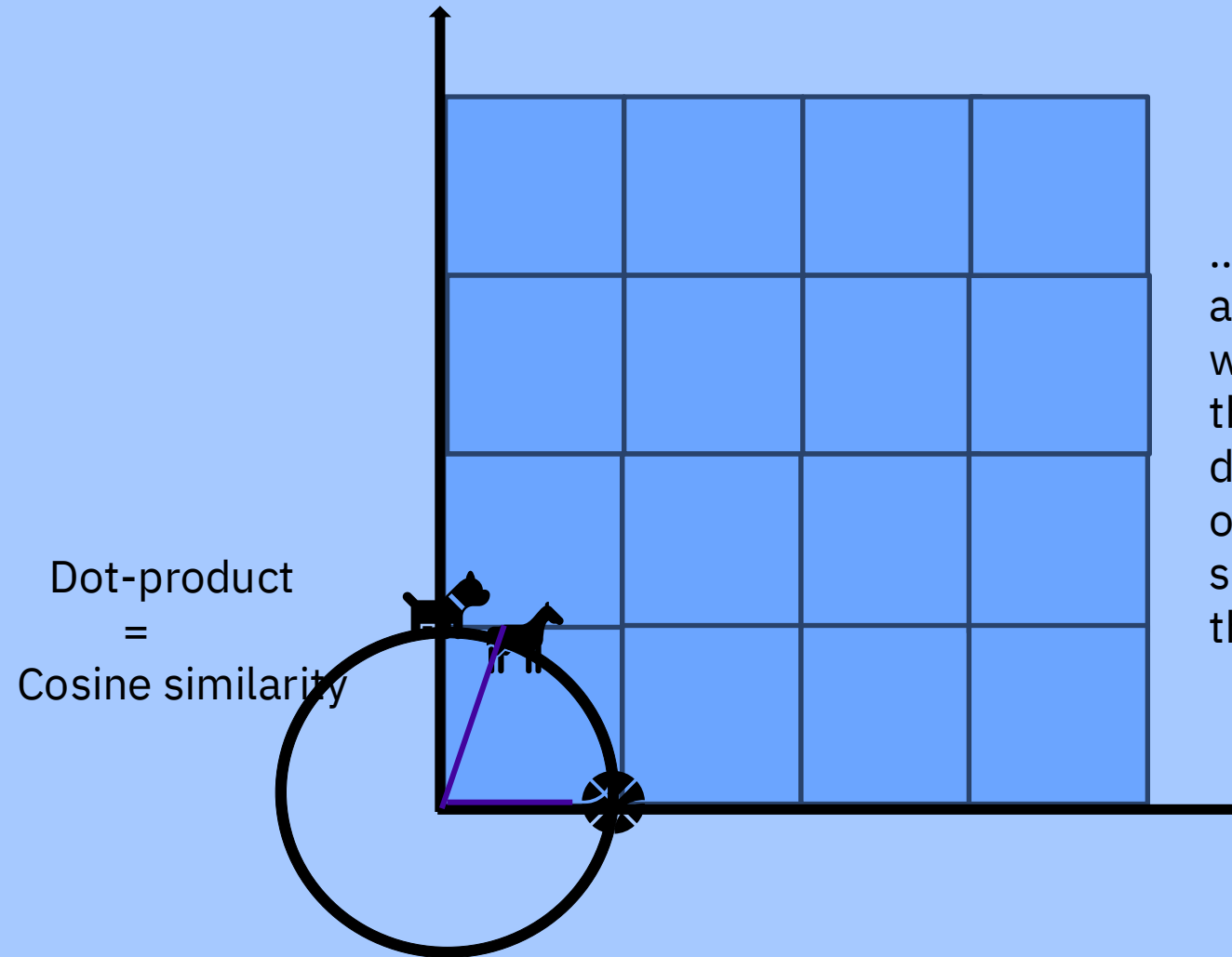


$$\cos(14) = 0.97$$

$$\cos(76) = 0.24 \quad \theta = \arctan(1/4) ; \text{deg} = 180/\pi$$

$$\cos(90) = 0$$

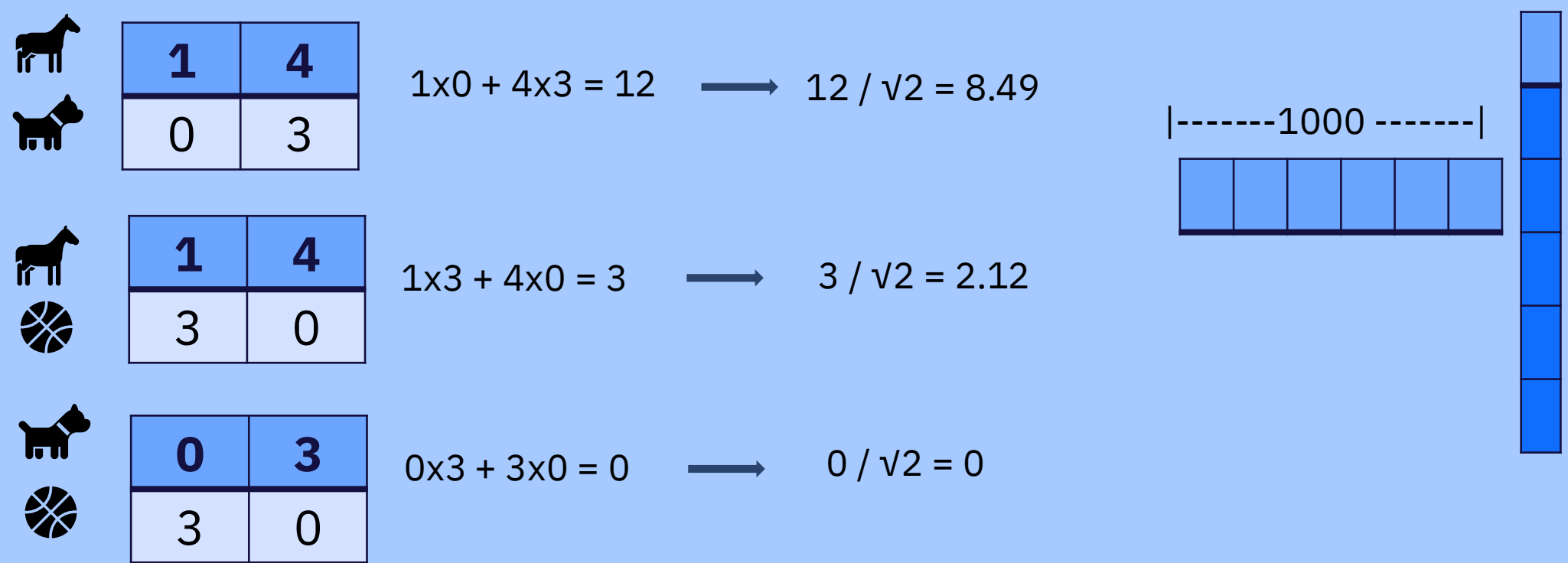
Now if vectors have length one...



...that is, if I draw a unit circle around the origin, and put the word, where it meets the circle, that means, I scale everything down so all vectors have length one, then dot-product and cosine similarity are the exact same thing!

Scaled dot product measure

Dot product divided by the **square root of the length of the vector**.
And this is the one that gets used in the Attention Mechanism!

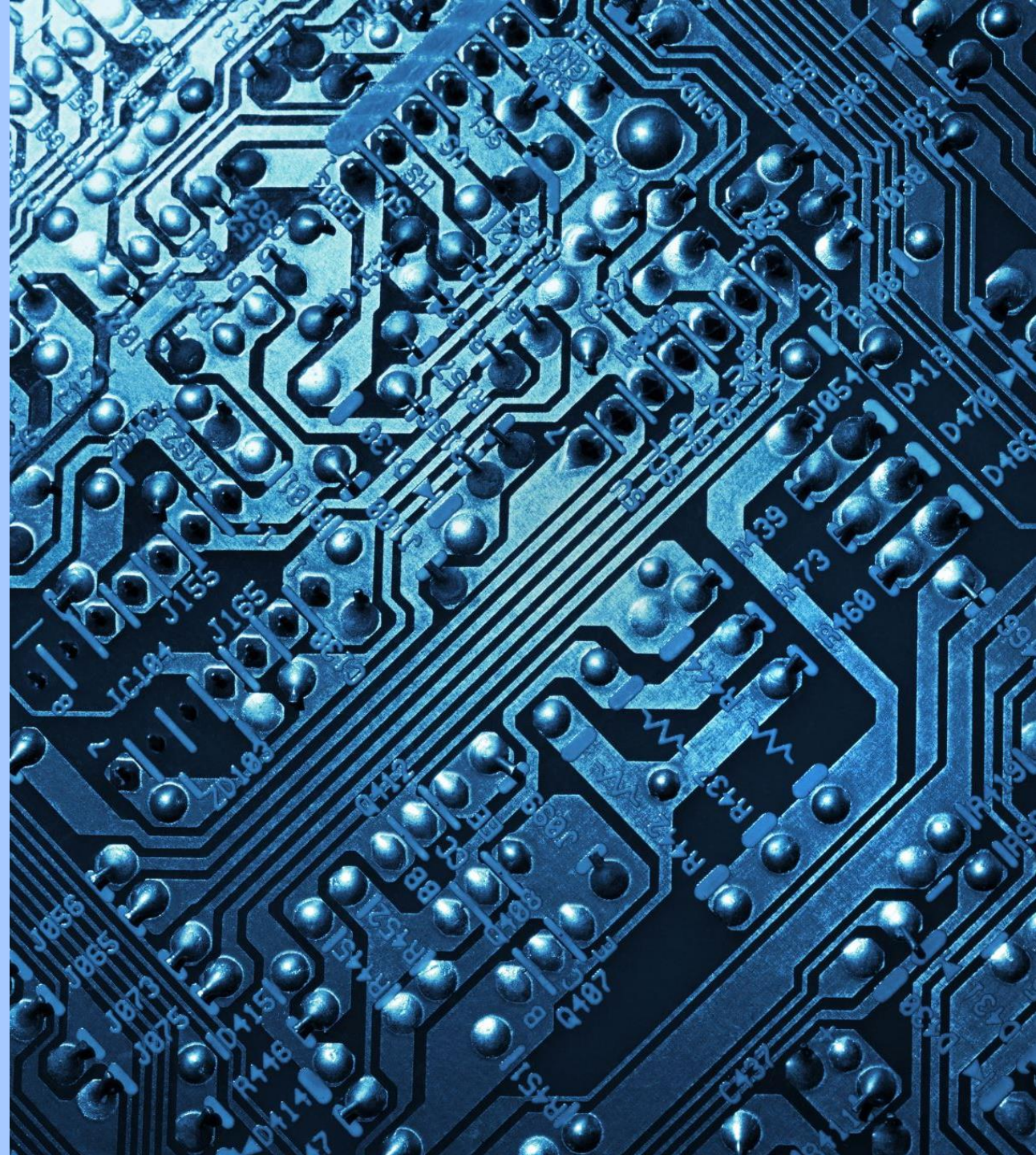


1. Overview of Gen AI
2. Attention is All you Need
3. It's About Cosine Similarity
4. **What's Next for Gen AI**
5. Is AI going to Replace Me?
6. Resources



Latest AI Trends in

1. Reality check: not just a chatbot, but rather integrated with other tools
2. Multi-modal: bring on text, images and video
3. Smaller is better: we are reaching the end of largeness in LLMs
4. GPU and Cloud costs:
5. Model optimization: LoRA
6. Custom local models: RAG
7. Agentic AI
8. Regulation: Performance, security, privacy, transparency
9. Shadow AI: unofficial use of AI at work and school
10. Your guess??



This is the Year of Reality Check...more Realistic Expectations

Shortly after Nov 2022, after ChatGPT was introduced to the masses, the craze was chatbots and Dall-E for vision.

This year many GenAI tools are being implemented as integrated elements, rather than stand-alone chatbots.

They enhance and complement existing tools rather than supplant them. Think Copilot features in MS Office or Generative Fill in Adobe Photoshop.

Embedding AI into everyday workflows helps us better understand what GenAI can and cannot do.

Multi-modal AI

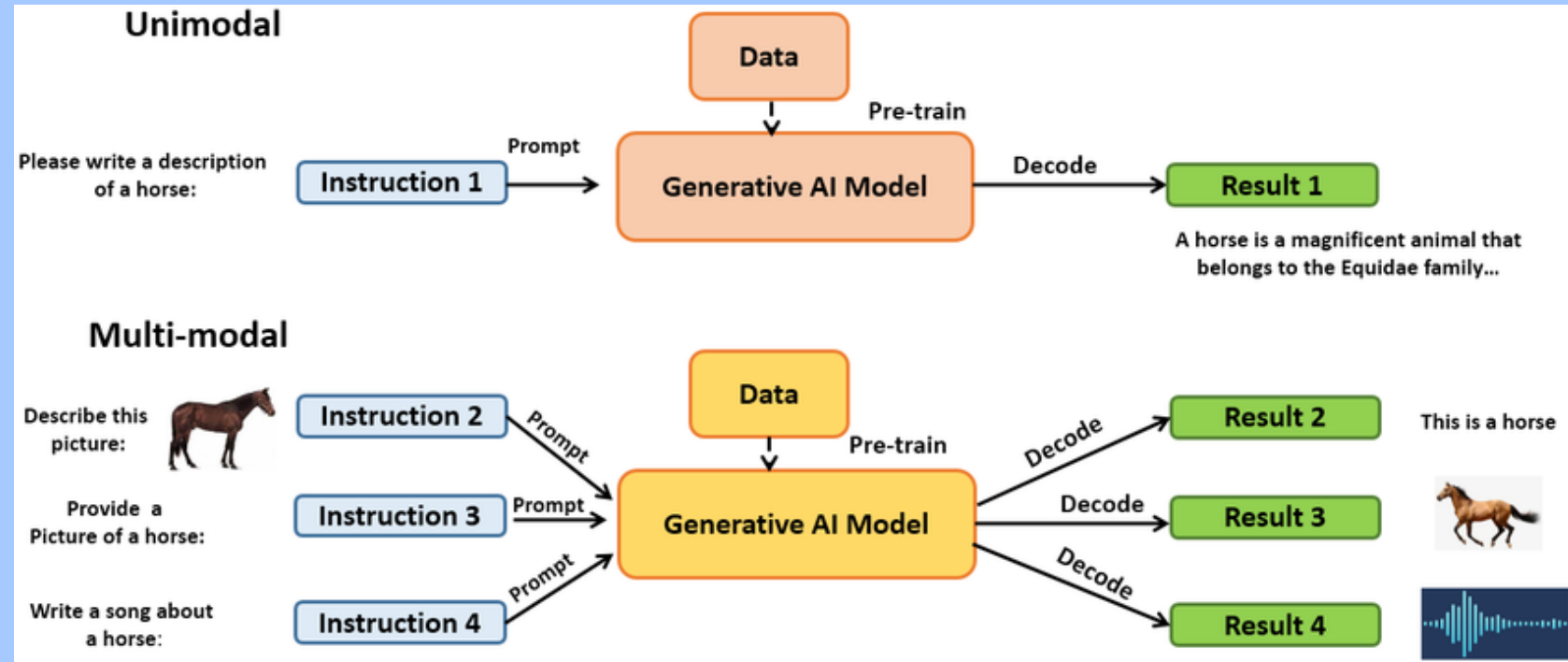
Multi-modal AI can take multiple layers of data as input.

We already have multidisciplinary models today like OpenAI's **GPT5.6** or Google **Gemini** that can move between NLP and Vision tasks.

New models are also bringing videos into the fold.

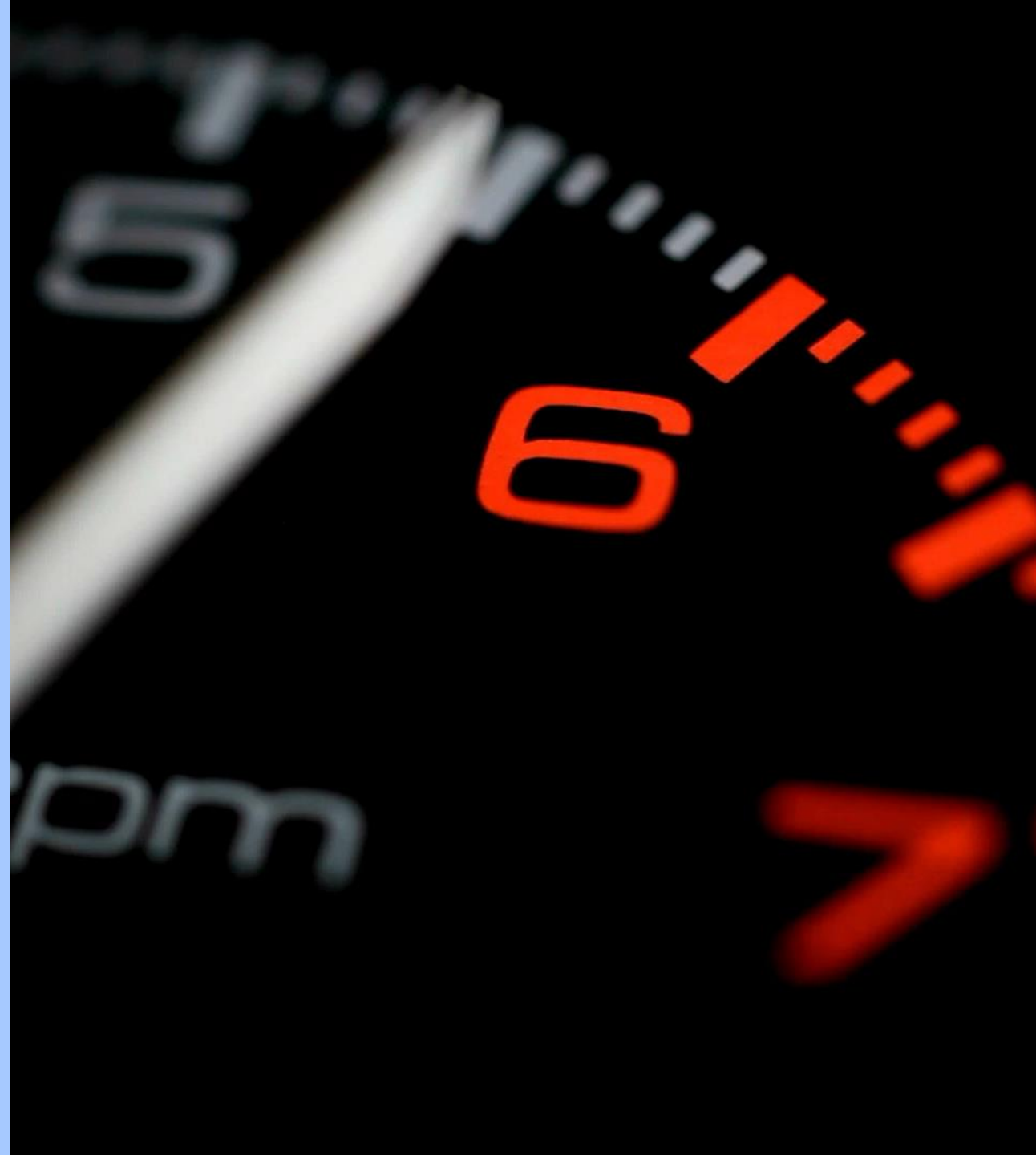
Self-driving cars rely heavily on multimodal AI.

We're on a path to create humanoids – systems that require human-like interactions or operations in environments designed for humans.



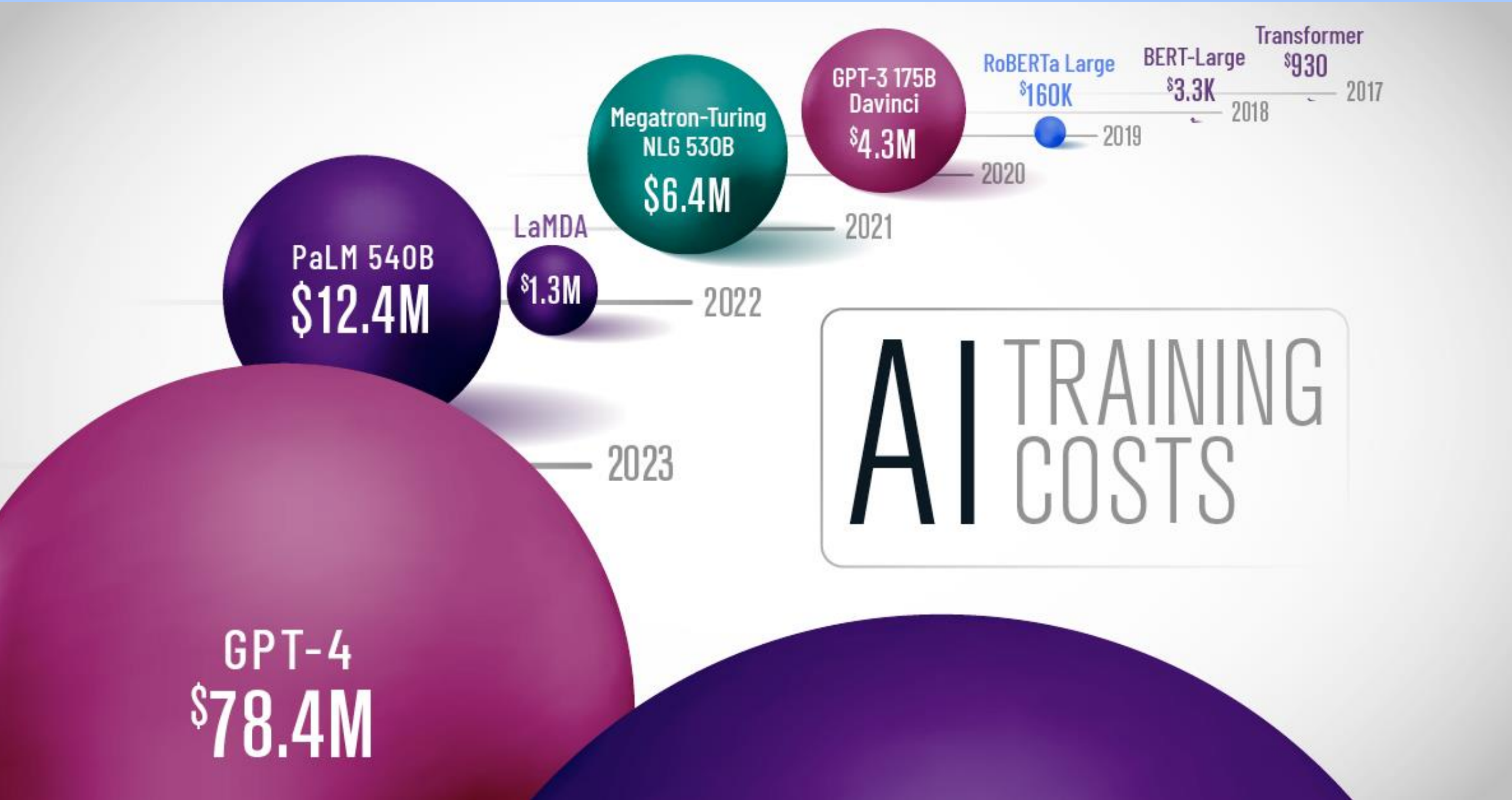
Unimodal vs Multimodal AI. Source: [ResearchGate](#)

Maximum AI with Minimum Data



Smaller is Better

- In the beginning, the craze was about how large is your LLM? And have you reached **trillion** parameters yet?
- Yes we have, but at an eye-watering costs: A recent study by the University of Washington, **training** a GPT3 size model requires the **yearly** electricity consumption of over **1000** households.
- So you may say, OK I get it, training large models does and should cost that much, but what about making an **inference**? That rivals the **daily** energy consumption of close to **33,000** households.
- The idea is to get greater output with fewer parameters. **GPT 4** is rumored to have around **1.76 trillion** parameters.
- Many **Open-source** models have great accuracy with model sizes in the **3 to 17 billion** parameter range.
- December of 2023, Mistral (French AI firm) released **Mixtral**. This is called Mixture of Experts, or **MoE** model: **8 Neural networks each with 7B parameters**. It outperforms the 70B parameter of Llama 2 at 6 times faster Inference speed and outperforms OpenAI's far larger GPT-3.5 on most benchmarks.



The expense is not sustainable

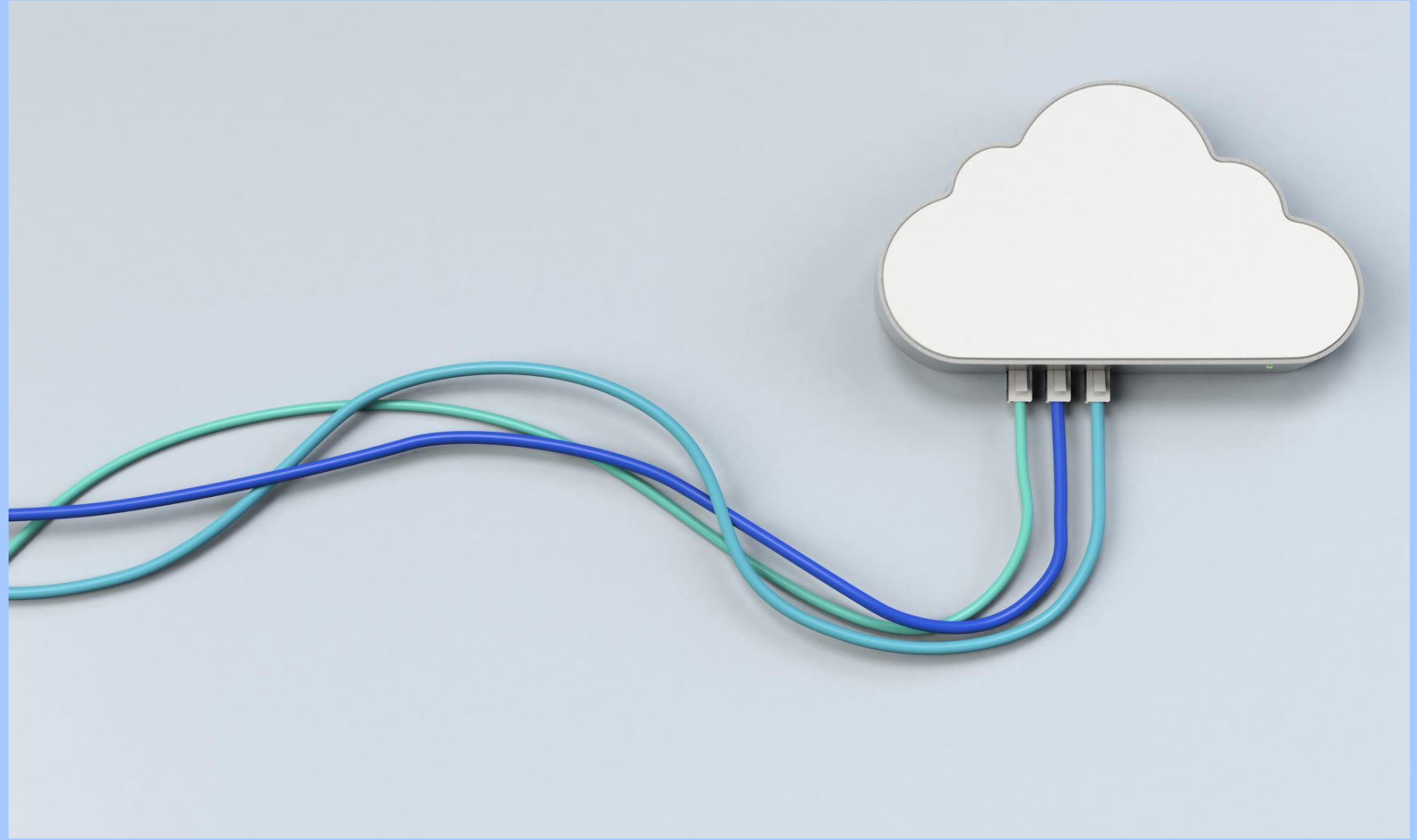
The larger the model the higher the requirements on GPU for training and inferences.

Everyone is scrambling to obtain the necessary GPUs to power the infrastructure, which puts upward pressure on Cloud costs.

2020: GPT3 175B \$4.3M

2023: GPT4 1.3T \$78.4M

What if these models were a bit more optimized...???



AI Model Optimization

This past year we have seen adoption of techniques for training, tweaking and fine-tuning pre-trained models like quantization.

For example reducing the file size of an audio file or video file by just lowering its bitrate.

In a similar vane, Quantization lowers the precision used to data points. For example, from a 16-bit floating point representation to an eight-bit integer to reduce memory usage and speed up inference.

...and then, there is LoRA: Low-Rank Adaptation. Freezing pre-trained model weights and injecting trainable layers in each Transformer block. LoRA reduces the number of parameters that need to be updated, which speeds up fine-tuning and reduces memory needed to store model updates

```
import numpy as np

# Original weight matrix W of size 6x4
W = np.array([
    [4, 1, 3, 2],
    [2, 3, 1, 4],
    [1, 2, 5, 3],
    [3, 4, 2, 1],
    [5, 3, 2, 4],
    [2, 1, 4, 3]
])

# Desired change matrix Delta W approximated by low-rank matrices A and B
A = np.array([
    [0.76, 1.19],
    [1.31, 0.83],
    [-0.36, 1.91],
    [1.39, 0.75],
    [1.93, 1.14],
    [-0.03, 1.61]
])

B = np.array([
    [1.72, 1.39, -0.38, 0.85],
    [1.15, 0.91, 2.53, 1.74]
])

# Compute the approximated change matrix Delta W
Delta_W_approx = np.dot(A, B)

# Updated weight matrix W'
W_prime = W + Delta_W_approx

# Print results
print("Original weight matrix W:\n", W)
print("\nApproximated change matrix Delta_W_approx:\n", Delta_W_approx)
print("\nUpdated weight matrix W_prime:\n", W_prime)

# Output:
Original weight matrix W:
[[4 1 3 2]
 [2 3 1 4]
 [1 2 5 3]
 [3 4 2 1]
 [5 3 2 4]
 [2 1 4 3]]

Approximated change matrix Delta_W_approx:
[[2.6757 2.1393 2.7219 2.7166]
 [3.2077 2.5762 1.6021 2.5577]
 [1.5773 1.2377 4.9691 3.0174]
 [3.2533 2.6146 1.3693 2.4865]
 [4.6306 3.7201 2.1508 3.6241]
 [1.7999 1.4234 4.0847 2.7759]]

Updated weight matrix W_prime:
[[6.6757 3.1393 5.7219 4.7166]
 [5.2077 5.5762 2.6021 6.5577]
 [2.5773 3.2377 9.9691 6.0174]
 [6.2533 6.6146 3.3693 3.4865]
 [9.6306 6.7201 4.1508 7.6241]
 [3.7999 2.4234 8.0847 5.7759]]
```

Key Terms Regarding LoRA

1.Weights: In a machine learning model, weights are the parameters that the model learns to adjust during training. They determine how input data is transformed into output predictions.

2.Matrix: A matrix is a grid of numbers with rows and columns. In neural networks, weights are often stored in matrices.

3.Independent: In the context of matrices, "independent" refers to the idea that the columns (or rows) of a matrix provide unique information. If columns are independent, no column can be formed by combining other columns.

4.Rank: The rank of a matrix is the number of independent rows or columns it has. For example, if a matrix has 3 columns but only 2 of them are independent, its rank is 2. Rank gives a sense of how much unique information is in a matrix.

5.Low-rank: A matrix has low rank when the number of independent columns (or rows) is much smaller than the total number of columns (or rows). Low-rank matrices are simpler and have fewer degrees of freedom.

It's called Rank Decomposition

The **rank** of a matrix is the number of **independent** rows or columns it has. It tells you how much unique information the matrix contains.

This is how rank decomposition is used in LoRA, where the focus is on simplifying the matrix while retaining the key information.

Understanding Matrix Rank

Rank = the number of linearly independent columns (or rows).

- A matrix has **rank 3** if all 3 columns are independent.
- A matrix has **rank 2** if one column can be built from the other two.
- A matrix has **rank 1** if all columns are multiples of a single column.
- In LoRA, low-rank updates keep only a few important directions.

Matrix rank = 3

$$A = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

3 independent columns

$$c_1 = (1, 0, 1)^T, c_2 = (0, 1, 1)^T, c_3 = (2, 1, 0)^T$$

No column can be written as a combination of the others.
All 3 columns are independent => rank = 3

Matrix rank = 2

$$B = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

column 3 depends on columns 1 and 2

$$c_1 = (1, 0, 1)^T, c_2 = (0, 1, 1)^T, c_3 = (1, 1, 2)^T$$
$$c_3 = c_1 + c_2$$

Only 2 columns are independent => rank = 2

Matrix rank = 1

$$C = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 1 & 2 & 3 \end{pmatrix}$$

all columns are multiples of c_1

$$c_1 = (1, 0, 1)^T$$
$$c_2 = 2c_1, \quad c_3 = 3c_1$$

Only 1 column is independent => rank = 1



Key idea: rank counts unique information. **Lower rank means more redundancy.**

Say hello to LoRA

Let's break down a simple **rank decomposition** using the Low-Rank Adaptation (LoRA) technique. We'll start with a matrix and perform a low-rank decomposition by breaking it into two smaller matrices, A and B , and show how they can approximate the original matrix W .

Example:

Suppose we have a simple matrix W , which is a 3×3 matrix (for simplicity), like this:

$$W = \begin{bmatrix} 6 & 2 & 4 \\ 1 & 3 & 5 \\ 7 & 8 & 9 \end{bmatrix}$$

Step 1: Low-Rank Decomposition

We'll approximate W using two smaller matrices, A and B , where A is 3×2 and B is 2×3 , representing a **rank-2** approximation. The rank-2 means we will reduce the complexity of the matrix while still capturing the essential structure.

Let's assume A and B are as follows:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 0 \end{bmatrix}$$

Step 2: Compute $A \times B$

Now, we multiply matrices A and B to get an approximation for W . The multiplication of a 3×2 matrix A with a 2×3 matrix B will result in a 3×3 matrix, like W .

$$\Delta W = A \times B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 2 \end{bmatrix} \times \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 0 \end{bmatrix}$$

Performing the matrix multiplication:

$$\Delta W = \begin{bmatrix} (1 \times 2 + 0 \times 1) & (1 \times 1 + 0 \times 2) & (1 \times 3 + 0 \times 0) \\ (0 \times 2 + 1 \times 1) & (0 \times 1 + 1 \times 2) & (0 \times 3 + 1 \times 0) \\ (1 \times 2 + 2 \times 1) & (1 \times 1 + 2 \times 2) & (1 \times 3 + 2 \times 0) \end{bmatrix}$$

$$\Delta W \downarrow \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 0 \\ 4 & 5 & 3 \end{bmatrix}$$

Step 3: Final Approximation of W

So, the matrix ΔW , which is the result of multiplying $A \times B$, is:

$$\Delta W = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 0 \\ 4 & 5 & 3 \end{bmatrix}$$

Now, if we started with a matrix W and applied a low-rank update in LoRA, the **new weight matrix** W_{new} would be:

$$W_{\text{new}} = W + \Delta W$$

Adding the two matrices element-wise:

$$W_{\text{new}} = \begin{bmatrix} 6 & 2 & 4 \\ 1 & 3 & 5 \\ 7 & 8 & 9 \end{bmatrix} + \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 0 \\ 4 & 5 & 3 \end{bmatrix}$$
$$W_{\text{new}} = \begin{bmatrix} 8 & 3 & 7 \\ 2 & 5 & 5 \\ 11 & 13 & 12 \end{bmatrix}$$

Conclusion:

We have applied **Low-Rank Adaptation** by:

1. Approximating a matrix W with two smaller matrices A and B .
2. The product of $A \times B$ gives a low-rank matrix ΔW , which is the update applied to the original matrix.
3. The result is a new matrix W_{new} that is more efficient to compute and store.

This illustrates the basic idea of how LoRA reduces the computational load by focusing on low-rank approximations of large matrices.

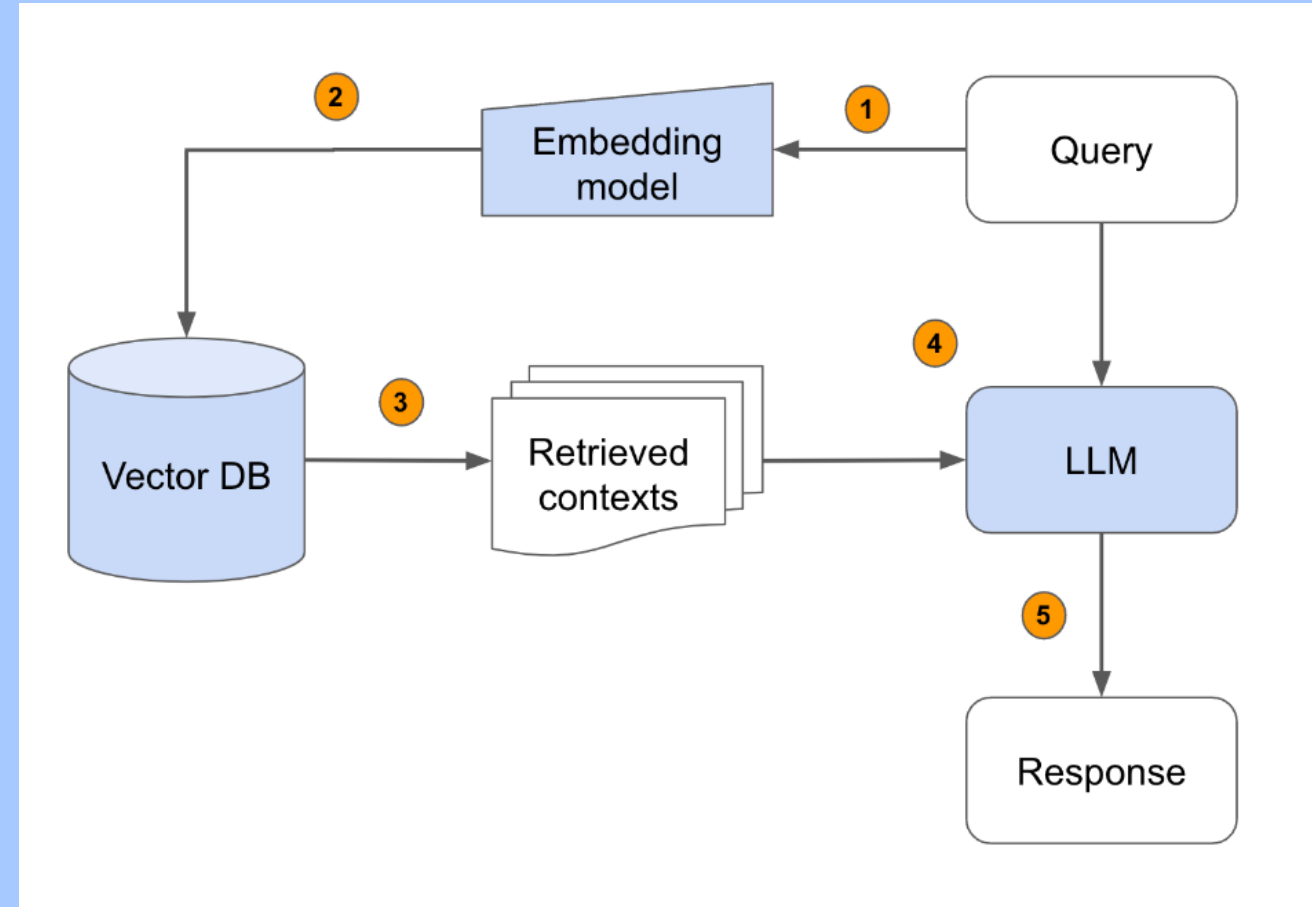
Custom Local Models - RAG

Open-source models lend well to developing powerful custom AI models.

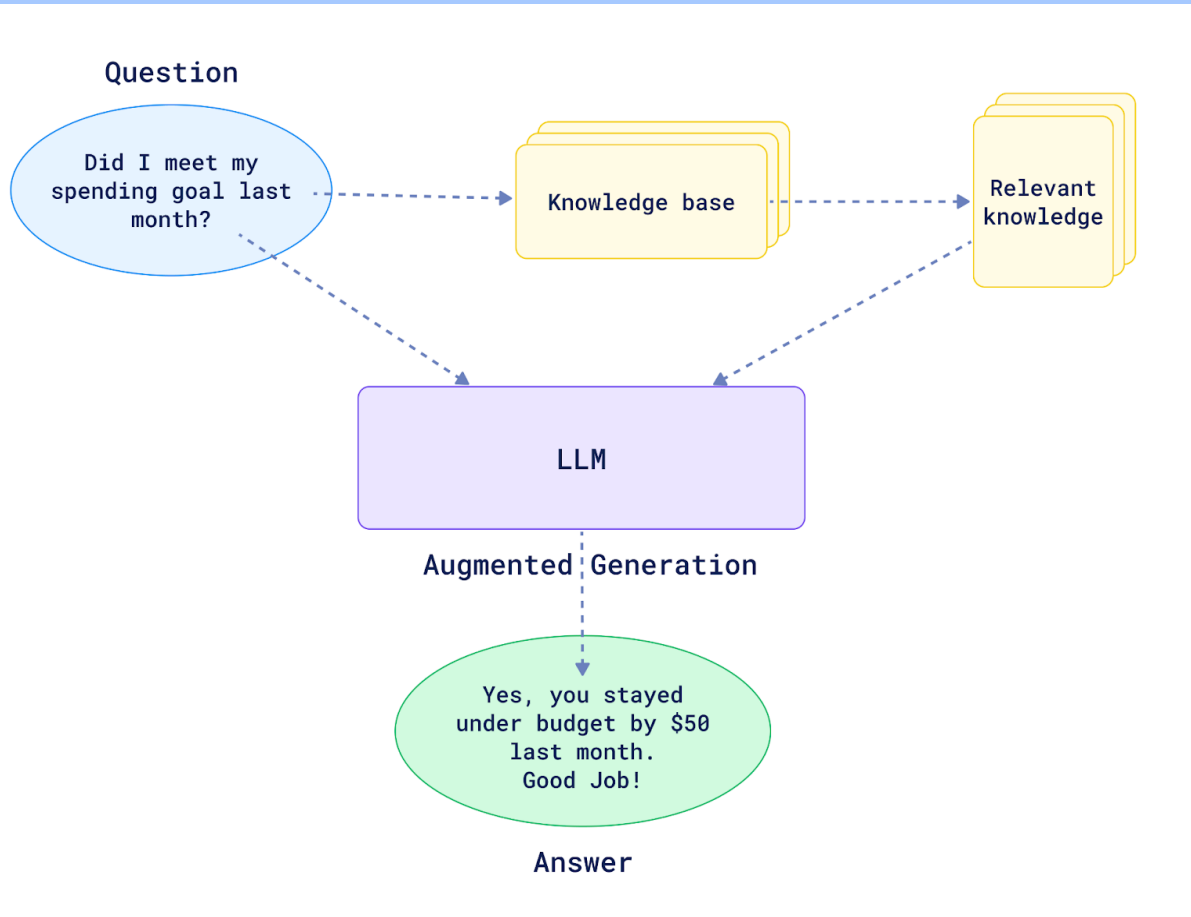
That means, trained on an organization's proprietary data and fine-tuned for their specific needs.

Tools such as RAG (Retrieval Augmented Generation) where you access company-specific information rather than storing it in a general-purpose publicly available LLM.

Sometimes the confidently given answer by the LLM is not so spot on. And it does not relay the source of its answer. That's where RAG comes in.



Retrieval Augmented Generation (RAG)



Retrieval Augmented Generation (RAG) is a method used in AI to help generate more accurate and helpful answers by combining two things: **retrieving** relevant information from uploaded content and using that information to **generate** (thanks to the LLM) a response.

The **uploaded document** serves as a knowledge base for the LLM, ensuring that answers are informed by this specific content and not just by its general training.

The **LLM** plays two major roles in RAG:

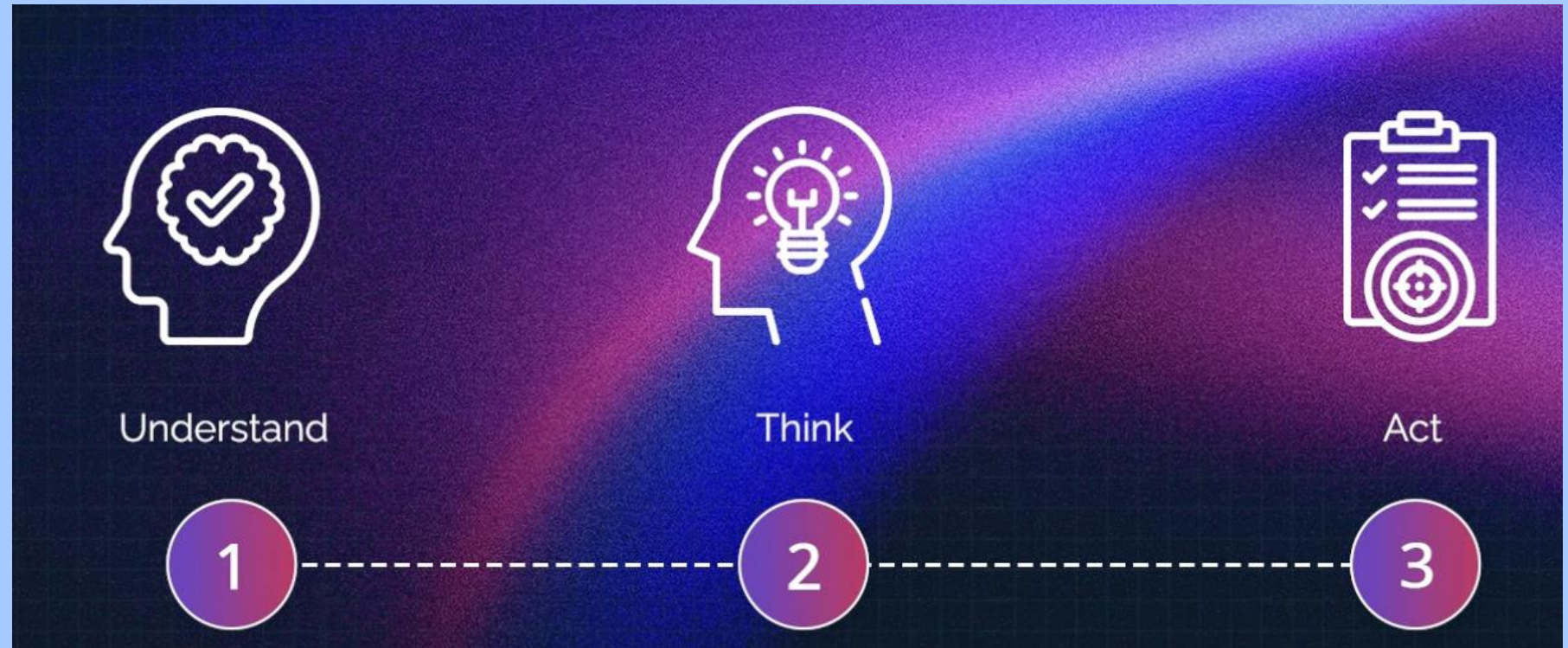
- 1. Understanding the Query:** The LLM is excellent at interpreting the question or request from the user, even if it's phrased in a complex or conversational way.
- 2. Generating a Response:** Once the LLM retrieves the relevant portions of the uploaded document, it combines that information with its own knowledge to generate a coherent, relevant response.

The AI Future is Agentic

It is more than just a chatbot answering your questions, but rather, agents carrying out tasks.

AI in 2024 is moving from monolithic models to Compound AI systems

LLMs will be connecting via APIs to LAMs (Large Action Models)



Features	RAG	AI Agents
Primary Focus	Knowledge Augmentation	Action and Interaction
Mechanism	Information Retrieval and Integration	Tool Utilization and Decision Making
Strengths	Improved Accuracy, Reliability, and Domain Expertise	Task Completion, Problem Solving and World Interaction
Limitations	Retrieval Performance, Static Context, Limited Interactivity	Tool dependency, Complexity of Agent Design, Ethical Considerations

AI Governance Framework

PERFORMANCE

ACCURACY

BIAS

COMPLETENESS

SECURITY

ADAPTABILITY

ADVERSARIAL
ROBUSTNESS

PRIVACY

IP CAPTURE

IMPACTED
USERS

TRANSPARENCY

EXPLAINABILITY

INTENT

Resources

MIT News – Explained: Generative AI

<https://news.mit.edu/2023/explained-generative-ai-1109>

Forbes – What Is Generative AI: A Super-Simple Explanation Anyone Can Understand

<https://www.forbes.com/sites/bernardmarr/2023/09/19/what-is-generative-ai-a-super-simple-explanation-anyone-can-understand/>

McKinsey – What is ChatGPT, DALL-E, and generative AI?

<https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-generative-ai>

Harvard Business Review – How Generative AI Can Augment Human Creativity

<https://hbr.org/2023/07/how-generative-ai-can-augment-human-creativity>

IBM – What is Generative AI?

<https://www.ibm.com/think/topics/generative-ai>